

实验原理：AES 的工作模式与它们的安全边界

—— ECB 为什么会被拼出来，CBC 为什么挡不住，GCM 靠什么拦住

这份文件在实验包里的位置

文件	负责讲什么
学生上机步骤.md	怎么做 —— 敲什么命令、该看到什么
实验指导书-AES的ECB模式安全性.md	做什么 —— 任务、报告、评分
本文	为什么 —— 原理展开，把这几种模式讲透

指导书第 1 节给的是够用的最小原理。本文是它的展开版：把 ECB / CBC / CTR / GCM 逐个拆开，说清楚每一种模式到底做了什么、因此能防住什么、又必然防不住什么。

读本文不需要任何前置材料，也不需要跑代码 —— 但配合任务六的实测输出看，效果最好。

0. 一页速览

先把结论摆出来，后面再逐条解释。

模式	一个分组怎么算	需要 IV/nonce	要填充	有完整性保护	本实验的搬运攻击
ECB	$ct[i] = E(pt[i])$	✗ 不需要	✓ 要	✗ 没有	✓ 完全成功
CBC	$ct[i] = E(pt[i] \oplus ct[i-1])$	✓ 需要（随机、不可预测）	✓ 要	✗ 没有 （填充校验不算）	⚠ 多数失败，前缀相同则成功
CTR	$ct[i] = pt[i] \oplus E(nonce \parallel i)$	✓ 需要（绝不重复）	✗ 不要	✗ 没有	⚠ 字节级可搬、可翻（本实验未直接测）
GCM	CTR 加密 + GHASH 认证标签	✓ 需要（绝不重复）	✗ 不要	✓ 有	✗ 被 InvalidTag 拒绝

一句话记住这张表的读法：

前三行的“有完整性保护”全是 ✗ —— 这不是巧合，是设计目标不同。ECB / CBC / CTR 只解决“别人看不见”，GCM 才同时解决“别人改不了”。

1. 起点：AES 只会做一件事

AES 是一个分组密码（block cipher）。它的接口小得可怜：

AES_加密(密钥 K, 16 字节明文) -> 16 字节密文
AES_解密(密钥 K, 16 字节密文) -> 16 字节明文

一次只吃 16 字节，也只吐 16 字节。没有"消息"的概念，没有长度概念，更没有任何"这段数据是不是被改过"的判断能力。

固定密钥 K 时，AES 加密是集合 $\{0,1\}^{128}$ 上的一个**置换**（——映射）：

- **单射**：不同的明文分组必然得到不同的密文分组；
- **满射**：任何 16 字节都可能是某个明文的密文；
- **可逆**：有密钥就能算回来。

这三条合起来就是分组密码安全性的全部内容 —— 也仅此而已。请注意它**没有**承诺什么：

✗ 它没有承诺"相同明文得到不同密文"（恰恰相反，见 3.3）
✗ 它没有承诺"密文被改过能被发现"
✗ 它没有承诺"两条消息的关系会被藏起来"

这一节是整份文档的根。本实验里所有的问题，追到最后都不是 AES 的问题，而是"我们向 AES 要了它从来没答应过的东西"。

2. 工作模式：把 16 字节的能力用到任意长度上

现实中的消息是 51 字节、3 KB、4 GB，不是 16 字节。于是必须规定一套"用法"：

1. 消息怎么切成块？
2. 每块怎么调用 AES？
3. 块与块之间有没有关系？
4. 怎么知道切得对、拼得对？

这套用法规定就叫**工作模式**（mode of operation）。它是**独立于 AES 的一层设计**，由使用者在代码里选择 —— `modes.ECB()`、`modes.CBC(iv)`、`AESGCM(key)` 就是这个选择。

🔥 责任划分（本实验最重要的认识）

层	负责	出了问题找谁
分组密码 (AES)	把 16 字节搅乱，让人没有密钥就算不回去	AES 至今没有被实用攻击
工作模式	把分组密码扩展到长消息， 并决定有没有完整性	本实验的问题全在这一层

说"ECB 被攻破了"是不准确的；准确的说法是 **"用 ECB 的系统被攻破了，而 AES 完好无损"**。

3. ECB：最简单的模式，也是最危险的

3.1 定义

ECB（Electronic Codebook，电码本模式）只有一行：

$$ct[i] = E(K, pt[i]) \quad i = 0, 1, 2, \dots$$

没有 IV，没有链式关系，没有计数器。每个分组都是一次完全独立的 AES 调用。最后一块不够 16 字节就补齐（本实验用 PKCS#7 填充）。

它有两个工程上很诱人的优点，也正是它至今仍潜伏在老系统里的原因：

- **可并行**：每个分组互不依赖，多核随便拆；
- **可随机访问**：想读第 7 个分组，直接解密第 7 个分组就行，不用从头算。

3.2 名字里的线索

"电码本" (codebook) 这个词不是随便起的。一本电码本就是一张**固定的对照表**：左边明文、右边密文，查表即得。

这个比喻精确得可怕——因为密钥固定时，ECB 加密**就**是一张表：一张 2^{128} 行的表，每一行都是"这个明文块 → 那个密文块"。 **同一行永远查得到同一个结果。**

于是所有的麻烦都从这一行字里长出来了。

3.3 漏洞一：确定性 → 模式泄露

性质 1 (确定性)：密钥固定时，**相同的明文分组必然产生相同的密文分组。**

这一条直接来自 1.1 的置换性质。后果是：加密没有隐藏消息的**结构**。

最直观的演示：ECB 企鹅

有一张流传很广的对照图：把一张企鹅照片的**像素**直接按 ECB 加密（像素数据被切成 16 字节一块一块加密），结果企鹅的轮廓清清楚楚——因为大片同色区域的像素块完全相同，加密后依然完全相同。

图像没有被"看见"，但被**认出来了**。加密本该隐藏的东西，ECB 原样交了出去。

本实验里的模式泄露（任务二实测）

本实验的消息格式 B 是"一个字段一个分组"。看这四条密文的分组 0：

消息	发件人	分组 0 (密文)
M1	Alice	08096a96a3a0475152d184e5931b4b1a
M2	Alice	08096a96a3a0475152d184e5931b4b1a
M3	Alice	08096a96a3a0475152d184e5931b4b1a
M6	Alice	08096a96a3a0475152d184e5931b4b1a
M4	Carol	8a75dc86c8b472ad42785f678d721fdb
M5	Bob	f72917986023578132aba98029dd28e7

四条完全相同的密文分组，攻击者一分钱没花，就知道这四条消息是同一个人发的。他甚至不需要知道这个人叫 Alice——他只需要知道"这四条的某个字段相同"。

同理：

- 分组 2 相同的 M1 / M2 / M4 / M6 → **同一天**；
- 分组 3 相同的 M1 / M3 / M6 → **同一个地点**；
- M1 和 M2 只有分组 3 不同 → **这两条只差地点**。

⚠ **一个必须澄清的误解** "模式泄露"不违反任何密码学定义。AES 依然是安全的伪随机置换。泄露的是**元数据**：谁、哪天、和谁 —— 而这些恰恰是情报价值最高的部分。安全从来不是"密文看起来像乱码"就够了的。

真实世界：2013 年 Adobe 数据泄露

这不是课堂假设。2013 年 Adobe 泄露的近 1.5 亿条账号记录里，口令是用 **3DES 的 ECB 模式** 加密的（不是哈希），所有记录共用同一个密钥，口令提示（hint）甚至是明文存放的。研究者做的事和你在任务二里做的事一模一样：

- 把密文**排序、计数** → 出现 191 万次的同一个密文，对应的明文是 123456；
- 密文分组相同 → 这些用户的口令相同（或前 8 字节相同）；
- 结合明文的提示字段 → 直接猜出大量口令。

整个分析**没有破解密钥**，密钥至今没有被恢复。ECB 把该藏的规律全抖了出来。

3.4 漏洞二：无完整性 → 密文分组可以当积木搬

性质 2（无完整性）：ECB 的密文分组之间没有任何绑定关系。

因为 $ct[i] = E(pt[i])$ ，每个分组独立加解密。解密端拿到 $ct[0]$ $ct[1]$ $ct[2]$ $ct[3]$ ，只会老老实实地解出四个明文块拼起来 —— 它**没有任何信息**可以判断 "这四块是不是来自同一条消息"。

于是攻击者可以：

- **调换**分组顺序；
- **删除**分组；
- **跨消息搬运**：把密文 Y 的第 3 块，接到密文 X 的第 2 块后面。

这就是本实验的攻击，学名 **密文分组拼接攻击**（ciphertext block splicing / cut-and-paste attack）。

关键在于：**做这件事完全不需要密钥**。攻击者不需要知道任何一块的明文，他只是在搬运 16 字节的内存。任务的输入和输出都是密文，中间过程没有一次解密 —— 这正是 `task4_forge.py` 连 `ecb_common` 都不 import 的原因。

3.5 漏洞三（本实验的发现）：搬运要成立，边界得对齐

分组搬运有一个**工程前提**，任务三故意让你先失败一次：

格式 A（自然写法）下，`From: Alice\nTo: Bob\n...` 的分组边界切在字段中间，第 3 分组里装着 `ary + 填充`，与另一条消息的第 3 分组（装着 `3-15\nPlace: Gym`）根本不是同一个字段。搬过来当然解出乱码。

要让攻击**干净利落**，分组边界必须恰好落在字段边界上。而格式 B 恰好做到了：

```

"From: " (6) + sender(10) = 16 <- 分组 0
"To: "   (4) + to(12)     = 16 <- 分组 1
>Date: " (6) + date(10)   = 16 <- 分组 2
"Place: "(7) + place(8)   = 15 <- 分组 3 的前 15 字节, PKCS#7 再补 1 字节

```

编码格式 B 的程序员做的是**正确的事**：定长对齐让解析简单、不易出 bug。但同一个决定，从攻击者视角看，等于把消息做成了一盒**边界清晰的乐高积木**——连猜都不用猜，第几块就是第几个字节。

🔥 这是本实验最值得带走的思维训练

局部看起来最合理的选择（定长对齐），放在整体里可能是最危险的。安全不是某一层的属性，而是**整个系统**的属性。

3.6 小结：ECB 错在哪

层	有没有问题
AES 算法本身	✅ 没有任何问题，至今没有实用攻击
工作模式 ECB	❌ 确定性 → 泄露结构
工作模式 ECB	❌ 无完整性 → 允许搬运、删除、重排
消息格式（编码层）	⚠️ 定长对齐放大了攻击的威力，但 不是 根本原因

结论：ECB 唯一站得住的用法，是加密“单个、高熵的、16 字节的数据”（例如安全地传输一个随机生成的密钥）——这种情况下根本不存在“两块可以比较”，确定性和搬运都无从谈起。

任何比这更复杂的场景，都不应该用 ECB —— 它连“看起来安全”都做不到。

4. CBC：给分组加一条链子

4.1 定义

CBC (Cipher Block Chaining, 密码分组链接) 想解决的就是 3.3 的确定性：既然“相同明文块 → 相同密文块”是祸根，那就在**加密前先把它搅乱**。

```

加密: ct[i] = E(K, pt[i] ⊕ ct[i-1])      ct[-1] := IV
解密: pt[i] = D(K, ct[i]) ⊕ ct[i-1]     ct[-1] := IV

```

IV (Initialization Vector, 初始化向量) 是一个 16 字节的初始值，作用是在链条的最前面补一个“假的前一块”。

```

pt[0] —⊕—> E —> ct[0] —┐—> 输出
      ↑                |
      IV                ⊕—> E —> ct[1] —> ...

```

4.2 它解决了什么

相同的明文分组不再直接对应相同的密文分组 —— 因为每一块的输入是 $pt[i] \oplus ct[i-1]$ ，而 $ct[i-1]$ 是“前面的历史”，位置不同、历史不同，输入就不同。

前提是 **IV 每次随机**。如果 IV 固定，那么这个性质立刻退化：

两条消息前缀相同 + IV 相同 $\rightarrow ct[0]$ 相同 $\rightarrow ct[1]$ 的输入相同 $\rightarrow ct[1]$ 也相同 $\rightarrow$ 一路相同下去。**CBC 在固定 IV 下，对相同前缀的泄露和 ECB 一模一样。**

这不是理论推演，本实验测到了它（见 4.3(b)）。

4.3 它没有解决什么

这是本节的重点。CBC 加的是**链接**，不是**完整性**。这两件事经常被混为一谈。

(a) 填充校验不是完整性校验

CBC 必须填充（因为要凑满 16 字节）。解密端解完后会检查 PKCS#7 填充是否合法，不合法就抛 `ValueError`：填充字节非法。看起来像个“防篡改机制”？**不是。**

- 它只能检验**最后一个分组**的尾部几个字节；
- 攻击者改动**中间**的分组时，它可能**碰巧通过**：随机字节恰好构成合法填充的概率约为 $1/256$ ；
- 通过了会怎样？Bob 拿到的是一条**肉眼可见的乱码消息**，但程序**不会报错**，它会高高兴兴地把乱码交给上层业务逻辑。

🔥 实验指导附录 B.5 的“对照 2”就是这个现象：改动中间分组，填充校验**通过了**，只是内容全乱。一个只会“偶尔拦住”的检查，从来就不该被当作安全机制。

(b) 前缀相同 $\rightarrow$ 搬运照样成功（本实验实测）

CBC 的链接让每个密文分组绑定它的**前驱**。那么：

如果攻击者能找到一个分组，它的前驱和目标位置的前驱**恰好相同**，搬过去就能正常解密。

本实验正是这种情形（任务六对照 3）：

M1 与 M2 前三个明文分组完全相同（同发件人、同收件人、同日期）

`-> ct1[2] = e1e8f042a2aa8819853f24c3bd5c0832`

`-> ct2[2] = e1e8f042a2aa8819853f24c3bd5c0832`

相等：True

搬分组 $\rightarrow$ CBC：解密成功

From: Alice / To: Bob / Date: 2026-03-15 / Place: Gym <- 改成了

CBC 并不阻止分组搬运。 它只让“前驱对不上”的情况失败。

⚠ **精确表述（别讲错了）** 实验里 CBC 之所以被绕过，有一个前提：**演示程序用了固定的全零 IV**，于是前缀相同的两条消息，密文前缀也相同。

如果换成**每条消息一个随机 IV**，这两条消息从分组 0 起就完全不同，上述搬运会失败。但这**不是**“CBC 有完整性保护”，理由见下面 (c)(d)：随机 IV 只是让攻击者更难凑齐“前驱相同”的条件，系统依然**没有任何能力判断密文是否被改过**。

(c) 比特翻转：不用搬运，直接改内容

这是比搬运更精细的攻击，而且和 IV 是否随机无关：

$$pt[i] = D(K, ct[i]) \oplus ct[i-1]$$

注意 $ct[i-1]$ 是异或进来的。攻击者改不了 $D(K, ct[i])$ （那要密钥），但他可以改 $ct[i-1]$ ——改一位， $pt[i]$ 的对应位就**确定地翻转一位**！

一次翻转会造成**两个**后果，必须都看清：

被改的是	后果
$pt[i-1] = D(ct[i-1]) \oplus ct[i-2]$	$D(ct[i-1])$ 变得完全不可预测 → 这一块变成乱码
$pt[i] = D(ct[i]) \oplus ct[i-1]$	对应比特 精准翻转 → 这一块可以被改成任意想要的值

所以 CBC 的比特翻转是**“牺牲一块，控制下一块”**：攻击者**不需要密钥，也不需要知道 $D(ct[i])$ 是什么**——他只要能猜出 $pt[i]$ 现在的值（比如 Date: 2026-03-15，格式已知），就能算出该异或多少，把它精确改成 Date: 2026-12-31。

代价是前一块变乱码。在本实验这种“四个分组全是有效字段”的紧凑格式里，这块乱码很显眼；但真实系统的格式里几乎总有**攻击者可以牺牲的块**——填充区、他不在乎的字段，或者先用 4.3(b) 的搬运把乱码挪到末尾。经典的真实攻击场景就是加密的 Cookie：

user=alice;role=user;...，翻几个字节把 role 改成管理员，被毁掉的那块恰好是攻击者不关心的部分。

CBC 的错误扩散（改一块毁两块）常被误当成“安全性”。它只是**把篡改变得难看**，没有把篡改变得**不可能**——而且“难看”的那一块，攻击者往往有权选择是哪一块。

(d) 填充预言攻击（更多拓展）

顺着 (a) 再走一步：如果系统对“填充非法”和“填充合法但内容错”给出**不同的反馈**（报错 vs 正常返回、不同的响应时间、不同的 HTTP 状态码……），攻击者就有了一个**预言机**。

他可以从最后一个分组开始，逐字节地把明文**解出来**——平均每个字节约 128 次询问，一个 16 字节分组大约两千次上下。这就是著名的 **padding oracle attack**（填充预言攻击，2010 年由 Juliano Rizzo 与 Thai Duong 公开，曾实战攻破 ASP.NET 的 WebResource.axd 与多个 TLS 实现）。

它的根因就是：CBC 的“合法性判断”被迫依赖填充校验，而填充校验是一个会泄露信息的旁路。

4.4 CBC 的正确打开方式

如果非要用 CBC（比如兼容既有系统）：

1. **IV 必须每次随机**，且不可预测——用 `os.urandom(16)`，绝不用固定值、绝不复用；
2. **必须再加完整性保护**，标准做法是 **Encrypt-then-MAC**：先 CBC 加密，再对密文（连同 IV）算 HMAC，解密方**先验 HMAC，再解密**；
3. **绝不要自己实现**“填充错误”与“内容错误”的区分逻辑——统一返回一类模糊错误。

或者干脆换掉：**新系统直接用 AEAD**（下一节）。

5. CTR：把分组密码当流密码用

理解 GCM 之前必须先理解 CTR —— 因为 **GCM 的加密部分就是 CTR**。

5.1 定义

CTR (Counter, 计数器模式) 不再加密明文，而是加密**计数器**：

```
ks[i] = E(K, nonce || counter_i)    <- 生成"密钥流"  
ct[i] = pt[i] ⊕ ks[i]                <- 明文与密钥流异或  
pt[i] = ct[i] ⊕ ks[i]
```

AES 在这里退化成一个**伪随机数发生器**：喂进去一个编号，吐出来 16 字节随机数。真正的加密动作只是异或。

5.2 它带来什么

性质	说明
不需要填充	密文长度 = 明文长度 (本实验任务六实测：63 字节)
可并行	密钥流可以提前批量算出来
可随机访问	想读第 100 块，直接算第 100 块的密钥流，不用从头解
错误不扩散	密文改 1 比特，明文只错 1 比特 (对比 CBC 的一块毁两块)

前三条都是工程优势。"错误不扩散"则是一把**双刃剑** —— 下面就是刀刃的另一侧。

5.3 死穴：nonce 绝不能重用

计数器序列 (nonce || counter) 一旦重复，**密钥流就重复**，而密钥流重复 = 回到"两次一密" (two-time pad)，古典密码学里最难补救的错误：

```
ct1[i] ⊕ ct2[i] = pt1[i] ⊕ pt2[i]    <- 密钥流被消掉了
```

两条密文一异或，剩下的就是两条**明文**的异或。如果其中一条的内容是可以猜的 (本实验里 Mallory 恰好知道历史消息长什么样)，另一条就直接暴露。

🔥 **本实验任务六就是这么干的**：两处 nonce 都写成 bytes(12) (全零)。这是刻意安排的反面教材 —— 正因为 nonce 相同，M2 的密钥流可以直接搬到 M1 的位置上用，GCM 的密文分组才能"搬得动"。

5.4 CTR 也防不住比特翻转 —— 而且更精准

```
ct[i] = pt[i] ⊕ ks[i]    =>   翻转 ct 的第 j 位    <=>   翻转 pt 的第 j 位
```

翻转任意一比特，明文的对应比特**必然**跟着翻转 (这一步不需要知道任何明文)；而如果攻击者知道该位置的当前值 (本实验里消息格式完全已知)，他就能把它**精确改成任意想要的值** —— 并且**没有 CBC 那种"牺牲一块"的代价**：

	CBC 比特翻转	CTR 比特翻转
被改的块	✅ 精准控制	✅ 精准控制
连带损伤	❌ 前一块变乱码	✅ 没有，一个字节都不多

CTR 的"错误不扩散"在工程上是优点，在这里对攻击者是**完美的工具**。

所以 CTR 单独使用时，完整性比 CBC 还要糟。

6. GCM: CTR + 一个认证标签

6.1 结构

GCM (Galois/Counter Mode) 是 AEAD (带关联数据的认证加密) 的代表，它由两半拼成：

AES-GCM = CTR (负责机密性)	+	GHASH (负责完整性)
↓		↓
密文和明文一样长		产出 16 字节认证标签

输出是 密文 || 标签。本实验任务六实测：明文 63 字节 → 密文 63 字节 + 标签 16 字节 = 79 字节。

6.2 标签是怎么算的

H	= E(K, 0 ¹²⁸)	<- 由密钥导出的 GHASH 密钥
J0	= nonce 0 ³¹ 1	<- 96 位 nonce 的标准构造
T	= GHASH(H, A, C) ⊕ E(K, J0)	<- 认证标签

其中 C 是全部密文，A 是**关联数据** (AAD，只认证不加密，比如消息头、路由信息)。

GHASH 是 $GF(2^{128})$ 上的**多项式求值**：把 A 和 C 的每一个分组当作多项式的系数，在点 H 上求值。这个构造有个关键性质：

❗ 改动任意一个分组的任意一比特 → GHASH 的结果完全不可预测地改变。

而多项式又要求**逐块**参与运算——这正是它和"搬运分组"针锋相对的地方。

6.3 为什么 GCM 能拦住本实验的攻击

因为**标签覆盖了整条密文**。攻击者搬动一个分组以后：

- 用**原来的标签** → 标签对不上新密文 → 解密抛 InvalidTag;
- 想算一个**新标签** → 需要 $H = E(K, 0^{128})$ → 需要密钥。

他没有第三条路。而"标签校验失败"和"填充错误"的根本区别在于：标签是一个**为完整性专门设计的、覆盖全消息的密码学校验值**，其伪造概率是 2^{-128} ；填充校验只是一个"块格式检查"，碰巧通过的概率约 1/256，而且它**从来没打算**检测篡改。

🔥 **这才是那个数字的意义**：1/256 与 2^{-128} 之间不是量级差别，是“根本没设计”和“专门设计”的差别。

6.4 但请注意：GCM 的分组层面同样挡不住搬运

这是任务六最反直觉的一条（对照 4 实测）：

密文部分 63 字节（= 明文长度，无填充）

GCM 的密文部分就是流密码： $ct[i] = pt[i] \text{ XOR } ks[i]$

****每个字节都只和自己位置上的密钥流有关**** — 所以照样可以搬。

Mallory 拿搬完的密文，配上他推算出的密钥流：

分组3 明文 = |Place: Gym | <- 分组层面搬成功了

GCM: 被拒绝 -> InvalidTag: 认证标签校验失败

⚠️ **精确表述** 任务六里 GCM 能搬，前提同样是**演示程序重用了同一个 nonce**（两处都是全零）。nonce 不重复时，搬过来的字节会解出乱码。但——**这不重要**。无论 nonce 重不重复、无论攻击者搬得动不动，**只要他没改标签，就会被拒；改了标签又算不出新的，还是被拒**。

真正拦住攻击的是**认证**，不是分组结构。“用了 AEAD 就安全”的重点在**认证**这两个字上。

6.5 GCM 的雷区：nonce 重用是灾难性的

CTR 里 nonce 重用已经很糟（密钥流泄漏）。在 GCM 里它更糟，因为它还会连累认证：

nonce 重用 → 两条消息的 $E(K, J_0)$ 相同、H 相同 → 攻击者拿到两条“同一 nonce、不同消息”的密文和标签，就有了关于同一个未知量 H 的**两个多项式方程**。解方程即可恢复 H（这就是 2016 年论文 *Nonce-Disrespecting Adversaries* 对多个 TLS 实现的实际攻击）。

而 H 一旦泄露，攻击者可以为任意消息伪造合法标签——机密性和完整性同时崩塌，不需要密钥。

使用 GCM 的三条铁律

1. **nonce 绝不重复**：推荐 96 位，用计数器或 `os.urandom(12)`；
2. 96 位 nonce 下单条消息不超过 $2^{32} - 2$ 个分组（约 64 GiB）；
3. 密钥一换，nonce 空间可以重新开始计数（**同一密钥下才要求不重复**）。

6.6 标签必须在解密之前校验

指导书思考题 5 问的就是这个。答案是：

一旦把（未认证的）明文交给上层，就已经晚了。

“先解密，再对明文算哈希比对”的问题是：解密已经发生，明文的每一比特都已经进入程序的内存、可能已经进入业务逻辑、日志、数据库、错误消息。真正要防的是**被污染的明文被当成数据使用**，而唯一能做到这一点的位置就是在**解密出口之前**。

这也是为什么 `AESGCM.decrypt()` 的设计是“校验失败就整个抛弃、只抛 `InvalidTag`”——它**不返回任何明文**。这个 API 形状本身就是安全设计的一部分，学名叫 **Authenticate-then-Decrypt**（先

认证，后解密）。

6.7 别把 GCM 当成万能

注意点	说明
AAD 只认证、 不加密	放进 AAD 的字段会被暴露（但被篡改会被发现）
标签可以变短，但要清楚代价	标签 t 位的伪造概率约 2^{-t} ；短标签（如 32 位）有实际被撞中的可能
nonce 空间不是无限的	见 6.5
它不解决“消息格式”问题	字段边界的泄露、格式本身的语义问题，GCM 一概不管

📌 **关于名字：**「GSM」是移动通信标准（Global System for Mobile Communications）的名字，与分组密码的工作模式无关——它的语音加密用的是另一套算法（A5 系列）。这里讲的模式叫 **GCM**，Galois/Counter Mode。

7. 横向对照总表

	ECB	CBC	CTR	GCM
核心式子	$ct[i]=E(pt[i])$	$ct[i]=E(pt[i] \oplus ct[i-1])$	$ct[i]=pt[i] \oplus E(nonce i)$	CTR + GHASH 标签
需要 IV/nonce	否	是（随机不可预测）	是（ 绝不重复 ）	是（ 绝不重复 ）
需要填充	是	是	否	否
可并行加密	✅	❌（加密时）	✅	✅
错误扩散	块内	一块毁两块	不扩散	不扩散
机密性	✅	✅	✅	✅
完整性	❌	❌	❌	✅
相同明文块 → 相同密文块？	✅ 是	IV 随机时否	否	否
密文膨胀	有填充	有填充	无	+16 字节标签
典型致命攻击	拼接、模式泄露	填充预言、比特翻转	nonce 重用	nonce 重用 （唯一的大坑）

本实验

任务六

攻击成功

前缀相同则成功

——

InvalidTag

结果

7.1 关于国密 SM4

一个自然的追问：**换成 SM4 是不是就好了？**

不是。**SM4 是分组密码 (GB/T 32907-2016)**，它和 AES 处在同一层——它的接口同样是“16 字节进、16 字节出”。所以：

- 用 **SM4-ECB** 加密本实验的消息 → **同样的攻击，一字不差地成功**；
- 用 **SM4-CBC** → 同样的填充问题、同样的比特翻转；
- 要安全，同样必须走 **SM4-GCM / SM4-CTR+HMAC** 这类带认证的组合，并且同样要管好 nonce。

算法自主与用法正确是两件事。用上国产算法是第一步；把它用对，才是安全的那一步。

8. 回到实验：六个任务分别在验证哪一条原理

任务	你观察到的现象	对应的原理
任务一	密文 = 16 字节一组的方块，可切片	分组密码的接口 (1.1)、PKCS#7 填充
任务二	4 条密文的分组 0 完全相同	漏洞一：确定性 → 模式泄露 (3.3)
任务三	先失败（乱码）后成功（日期被改）	漏洞三：搬运需要边界对齐 (3.5)
任务四	不用密钥伪造出 Place: Gym	漏洞二：无完整性 → 分组搬运 (3.4)
任务五	四块来自四条消息，拼成通顺的新消息	漏洞一 + 漏洞二 + 格式对齐的合力
任务六	CBC 被拒 / CBC 前缀相同则成功 / GCM 被拒	链接 ≠ 完整性 (4.3)、 认证才是关键 (6.3–6.4)

9. 三条可以带走的结论

1. **加密 ≠ 安全。** 机密性（别人看不见）和完整性（别人改不了）是**两件事**，必须**分别**获得。ECB / CBC / CTR 只给了第一件。
2. **模式的选择是安全决策，不是实现细节。** 同一个 AES，换个 `modes.xxx()` 就从“可被伪造”变成“伪造概率 2^{-128} ”。而这行代码通常没人会 review。
3. **密码学原语的安全，不等于系统的安全。** 本实验的攻击没有触碰 AES 一比特——它攻击的是**格式设计、模式选择**和**缺失的认证**。真实的密码学工程，大部分功夫都在这里。